4 load and explode the data set of csv

```
   import pandas as pd
   csv_data=pd.read_csv("C:/Users/Adith_aued46p/OneDrive/Desktop/data.csv")

   import pandas as pd
   excel_data =
pd.read_excel("C:/Users/Adith_aued46p/OneDrive/Documents/data.excel.xlsx")
    print(excel_data)


    import pandas as pd
    csv_data=pd.read_csv("C:/Users/Adith_aued46p/OneDrive/Desktop/data.csv")
     print(csv_data.describe())

    print("\nData Types in csv File:")


     print(data_csv.describe())
```

5 Write a program to Visualize the dataset to gain insights using Matplotlib by plotting scatter plots, bar charts.

```
     import pandas as pd
import matplotlib.pyplot as plt

# Sample Dataset
data = {
    'Student': ['A', 'B', 'C', 'D', 'E'],
    'Marks': [78, 85, 92, 67, 88],
    'Study_Hours': [3, 4, 6, 2, 5]
}

df = pd.DataFrame(data)

# Scatter Plot
plt.figure(figsize=(6,4))
plt.scatter(df['Study_Hours'], df['Marks'])
plt.title('Study Hours vs Marks')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.grid(True)
plt.show()

# Bar Chart
plt.figure(figsize=(6,4))
plt.bar(df['Student'], df['Marks'])
plt.title('Student Marks Comparison')
plt.xlabel('Students')
plt.ylabel('Marks')
plt.show()
```

6 Write a program to Handle missing data, encode categorical variables, and perform feature scaling.

```
     import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

# dumy Dataset
data = {
    'Age': [22, 25, None, 30, 28],
```

```
    'Gender': ['Male', 'Female', 'Female', 'Male', None],
    'Salary': [25000, 30000, 28000, 35000, 32000]
}

# ----------------------------
# Handling Missing Values
# ----------------------------
df_missing = df.copy()

# Fill missing Age with mean
df_missing['Age'] = df_missing['Age'].fillna(df_missing['Age'].mean())

# Fill missing Gender with mode
df_missing['Gender'] = df_missing['Gender'].fillna(df_missing['Gender'].mode()
[0])

print("\nData After Handling Missing Values:")
print(df_missing)

# ----------------------------
# Categorical Encoding
# ----------------------------
df_encoded = df_missing.copy()

encoder = LabelEncoder()
df_encoded['Gender'] = encoder.fit_transform(df_encoded['Gender'])

print("\nData After Categorical Encoding:")
print(df_encoded)

# ----------------------------
# Feature Scaling
# ----------------------------
df_scaled = df_encoded.copy()

scaler = StandardScaler()
df_scaled[['Age', 'Salary']] = scaler.fit_transform(
    df_scaled[['Age', 'Salary']]
)

print("\nData After Feature Scaling:")
print(df_scaled)
```

7 Write n program to implement a k-Nearest Neighbours (k-NN) classifier using scikitlearn and Train tho classifer on the dataset and evaluate its performance.

```
  import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Dummy student data: Exam Score 1, Exam Score 2
X = np.array([
    [80, 75],
    [95, 90],
    [68, 50],
    [45, 30],
    [38, 40],
    [85, 95],
    [70, 60],
    [58, 55],
    [40, 45],
    [60, 70]
```

```
])

# Target variable: 1 = Pass, 0 = Fail
y = np.array([1, 1, 0, 0, 0, 1, 1, 0, 0, 1])

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Initialize k-NN classifier with k = 3
knn = KNeighborsClassifier(n_neighbors=3)

# Train the classifier
knn.fit(X_train, y_train)

# Predict on test data
y_pred = knn.predict(X_test)

# Evaluate the classifier
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy on the test set: {:.2f}".format(accuracy))

# Take user input
exam_score1 = float(input("Enter Exam Score 1: "))
exam_score2 = float(input("Enter Exam Score 2: "))

# Prepare input for prediction
user_input = np.array([[exam_score1, exam_score2]])

# Predict outcome
predicted_outcome = knn.predict(user_input)

if predicted_outcome[0] == 1:
    print("Based on the exam scores provided, the student is predicted to
PASS.")
else:
    print("Based on the exam scores provided, the student is predicted to
FAIL.")
```

8 Write a program to implement a linear regression model for regression tasks and Train the model on a dataset with continuous target varÃables.

```
  import numpy as np
from sklearn.linear_model import LinearRegression

# Training data
X = np.array([
    [1000, 2],
    [1500, 3],
    [1200, 2],
    [1800, 4],
    [900, 2],
    [2000, 3]
])

y = np.array([300000, 400000, 350000, 500000, 280000, 450000])

# Create and train the model
model = LinearRegression()
model.fit(X, y)

# User input
```

```
size = float(input("Enter the size of the house in sqft: "))
bedrooms = int(input("Enter the number of bedrooms: "))

# Prediction
new_data = np.array([[size, bedrooms]])
predicted_price = model.predict(new_data)

# Output
print("Predicted price for a house with size {} sqft and {} bedrooms: Rs.{:.2f}"
      .format(size, bedrooms, predicted_price[0]))
```

9 Write a program to implement a decision tree classifer using scikit-learn and visualizc the decision tree and understand its splits.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
import matplotlib.pyplot as plt

# Custom dummy data for fruit classification
# Features: [Weight, Texture]
# Texture: 0 = Smooth, 1 = Rough

X = np.array([
    [150, 0],
    [170, 1],
    [120, 0],
    [140, 1],
    [200, 1],
    [130, 0]
])

# Target labels
y = np.array([
    'Apple',
    'Orange',
    'Apple',
    'Orange',
    'Melon',
    'Apple'
])

# Initialize the Decision Tree Classifier
clf = DecisionTreeClassifier(random_state=42)

# Train the classifier
clf.fit(X, y)

# Display decision tree rules
tree_rules = export_text(
    clf,
    feature_names=['Weight', 'Texture']
)

print("Decision Tree Classifier Rules:\n")
print(tree_rules)

# Plot the Decision Tree
plt.figure(figsize=(10, 6))

plot_tree(
    clf,
    filled=True,
    feature_names=['Weight', 'Texture'],
    class_names=np.unique(y)
```

```
)

plt.show()
```

10 Write a program to Implement K-Means clustering and Visualize clusters

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Generate dummy customer data
# Features: [Age, Income]

X = np.array([
    [30, 50000],
    [35, 60000],
    [40, 80000],
    [25, 30000],
    [45, 100000],
    [20, 20000],
    [50, 120000],
    [55, 150000],
    [60, 140000],
    [28, 40000]
])

# Initialize K-Means with 3 clusters
kmeans = KMeans(n_clusters=3, random_state=0)

# Train the model
kmeans.fit(X)

# Get cluster labels and cluster centers
labels = kmeans.labels_
centers = kmeans.cluster_centers_

print("Cluster Labels:")
print(labels)

print("\nCluster Centers:")
print(centers)

# Visualize the clusters
plt.figure(figsize=(8, 6))

plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels,
    cmap='viridis',
    s=100,
    label='Customers'
)

plt.scatter(
    centers[:, 0],
    centers[:, 1],
    marker='X',
    s=300,
    color='red',
    label='Centroids'
)
```

```
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('K-Means Clustering of Customers')
plt.legend()

plt.show()
```